Assignment 1 — How Matrix Size Affects a Basic Matrix Multiplication in Python, Java and C

Big Data · Grado en Ciencia e Ingeniería de Datos · ULPGC 2026-2027 · Individual Assignment 1 · Andrea Patruno · October 2026

Abstract. We measure how matrix size affects runtime and memory of the same triple-loop matrix multiplication (float64, same loop order and layout) in Python, Java and C, on shared inputs and one machine, measured with the course benchmarking tools (JMH, pytest-benchmark, perf) with 5 timed repetitions per configuration (median and IQR). All three produce identical results. Time grows as n^3 only while the matrices fit in cache (×38 instead of ×8 from n = 256 to 512 in C). Python is 48–161× slower than C and reaches only n = 512 in a 20 s budget; C and Java reach n = 1024, where Java is unexpectedly 1.2× faster. Conclusions hold only for these implementations and this machine.

1. Introduction and objective

Matrix multiplication is a basic Big Data kernel (graphs, machine learning, scientific computing) and its cost grows quickly with size. Before optimising it (later assignments) we need a controlled baseline.

Research question. How does the matrix size n affect the runtime and the memory use of the same basic triple-loop algorithm implemented in Python, Java and C?

- *Factor changed:* matrix size n (and the language).
- *Outcomes measured:* time per multiplication (median and IQR) and peak resident memory.

Hypotheses (stated before running).

- H1: the time grows as n^3 (×8 when n doubles) for all three languages.
- H2: C is the fastest, Java is close after JIT warm-up, Python is one to two orders of magnitude slower because it is interpreted.
- H3: C uses about $24n^2$ bytes for the three matrices plus a small runtime; Java adds the JVM (tens of MB); Python needs more memory per element because each float is an object.

2. Background

Algorithm. For C = A·B with n × n matrices, the straightforward method computes every C[i][j] as the dot product of row i of A and column j of B: three nested loops, n^3 multiply-add operations, $O(n^3)$ time and $O(n^2)$ memory [1].

Memory hierarchy. With row-major storage, the inner loop reads A[i][k] contiguously but B[k][j] with a stride of n·8 bytes, so each access to B can need a new cache line. When B no longer fits in the cache the cost per operation increases; with power-of-two strides many lines also map to the same cache sets (conflict misses) [2].

Languages. C is compiled ahead of time to machine code. Java is compiled to bytecode and then compiled at run time by the JIT, so the first runs are slower (warm-up).

CPython interprets bytecode and each float is a separate heap object, so every operation in the inner loop costs much more than one machine instruction.

3. Methodology

3.1 Implementations

The three programs (`src/`) contain the same loop, in the same order, on the same layout:

```
for i in 0..n-1:  for j in 0..n-1:  s = 0;  for k in 0..n-1:  s += A[i*n+k] *
B[k*n+j];   C[i*n+j] = s
```

	Python	Java	C
Storage	flat `list` of Python floats	flat `double[]`	flat `double*` (malloc)
Numeric type	float64 (boxed objects)	double (primitive)	double
Execution	CPython 3.14.4 interpreter	OpenJDK 21.0.12, JIT (C2), `-Xmx2g`	GCC 15.2.0 `-O2 -std=c11`
Benchmark tool	pytest-benchmark 5.2.3 [7]	JMH 1.35 [6] (Maven)	`perf stat` 7.0 [9]

No library call replaces the loops. Unavoidable differences: Python stores pointers to float objects (≈ 32 bytes per element instead of 8) and checks types at every operation; Java checks array bounds; C uses `-O2` without `-ffast-math`, so the compiler cannot reorder the floating-point sum [8] and all three perform exactly the same additions in the same order.

3.2 Inputs

`scripts/gen_inputs.py` writes A and B once for every n as binary files (float64, little-endian, row-major), and the three programs read the same files. Generator: NumPy PCG64 [4], seed 42 + n, values uniform in [−1, 1); A is drawn first, then B.

3.3 Environment

Item	Value
Machine	MSI laptop, Intel Core i7-1255U (10 cores: 2 performance + 8 efficiency, 12 threads), Windows host
Execution environment	WSL2, Ubuntu 26.04.1 (kernel 6.6.87.2-microsoft-standard-WSL2), 6 cores / 12 logical processors visible, 10.7 GiB RAM
Caches (per core, as reported in WSL2)	L1d 48 KiB, L2 1.25 MiB, L3 12 MiB (shared)
Threads	all three programs are single-threaded; no numerical library is used in the timed code

3.4 Correctness (before benchmarking)

`scripts/verify.py` runs the three programs on: a 2×2 case with known result, A·I = A and I·A = A (n = 50), A·0 = 0 (n = 50), and random matrices with n = 64 and n = 100

compared with NumPy `matmul` (reference only). Known-answer cases must match exactly. For random inputs $|a_{ik}|, |b_{kj}| \le 1$, so the floating-point error of a length-n dot product is at most $\approx n^2u$ per entry ($u = 2^{-53}$) [3]; we use absolute tolerance $n^2 \cdot \text{eps}$ (eps $= 2^{-52}$, which also covers the reference's own error) and no relative tolerance.

3.5 Measurement protocol

We use the benchmarking tools of the course for each language [6], [7], [9]:

- **Java — JMH:** `Mode.AverageTime`, 3 warm-up and 5 measurement iterations of 1 s, 1 fork with `-Xmx2g`. Each measurement iteration gives the average time per multiplication (for n ≥ 512 one iteration contains a single multiplication). The result array is returned to JMH, so it is consumed.
- **Python — pytest-benchmark:** `benchmark.pedantic` with 1 warm-up round and 5 rounds; each round runs 64 (n = 64), 8 (n = 128) or 1 multiplication, and the tool reports the time per multiplication.
- **C — perf:** the program, compiled with `gcc -O2`, runs under `perf stat`. Because `perf stat` measures the whole process (including file loading), the time of each repetition is taken inside the program with `clock_gettime(CLOCK_MONOTONIC)` around the kernel (1 warm-up, 5 measured, same batching as Python); `perf` records task-clock and the hardware counters (cycles, instructions, cache references and misses).
- **Timing boundary:** kernel only for all three. Loading the files and allocating A, B, C are outside the timed region.
- **Result consumed:** the checksum (left-to-right sum of C) is saved with every configuration.
- **Summary:** median, Q1 and Q3 (inclusive method), IQR = Q3 − Q1. All raw values are in `results/raw_times.csv`; the native output of each tool is in `results/tools/`.
- **Memory:** peak resident set size (VmHWM in `/proc/self/status` [5]) of the process that runs the kernel (JMH fork JVM, pytest process, C program), read at the end of the run. It includes the runtime (JVM and JMH, interpreter and pytest), so it is reported next to an estimate of the array storage: $3 \cdot 8n^2$ bytes for C/Java, $3 \cdot 32n^2$ for Python.
- **Sizes and budgets** (declared before running): n = 64, 128, 256, 512, 1024, 2048. Budget: 20 s for one multiplication, 2 GB peak memory. When 8 × the previous median exceeds 20 s, one multiplication is first attempted with a 20 s timeout (tool start-up included); if it times out, larger sizes are not attempted.

4. Results

4.1 Correctness

All tests passed for the three implementations. The known-answer, identity and zero cases match exactly (error 0). Against NumPy the maximum absolute error is 0 for both n = 64 (tolerance $9.1 \cdot 10^{-13}$) and n = 100 (tolerance $2.2 \cdot 10^{-12}$): on this machine the reference produced the same values. The outputs of Python, Java and C are **bitwise identical** in every test, and the benchmark checksums are identical across languages for every n.

4.2 Runtime

Table 1 — Time per multiplication (seconds), median of 5 repetitions with IQR. "—" = not attempted after a timeout at a smaller size.

n	C median	C IQR	Java median	Java IQR	Python median	Python IQR	Java / C	Python / C
64	0.0000699	0.0000012	0.000148	0.000003	0.0113	0.0012	2.1	161
128	0.00101	0.00004	0.00133	0.00002	0.0881	0.0077	1.3	87
256	0.00841	0.00022	0.0115	0.0001	0.972	0.107	1.4	116
512	0.317	0.005	0.363	0.017	15.08	0.31	1.1	48
1024	5.76	0.21	4.80	0.35	timeout		0.83	—
2048	timeout		timeout		—			

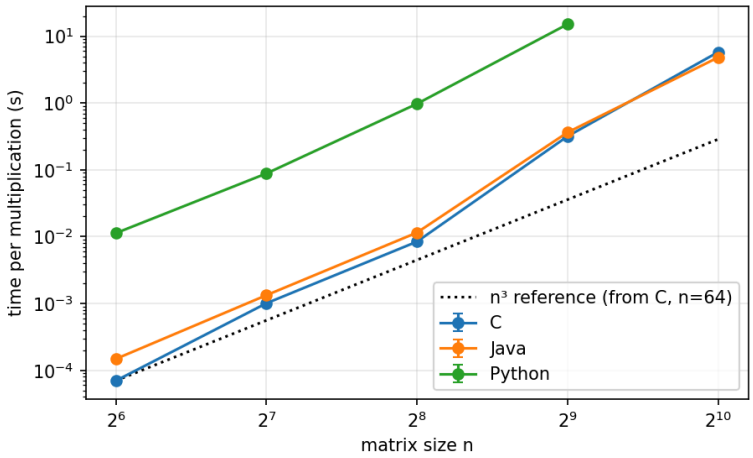


Figure 1 — Median time per multiplication vs n (log-log, error bars = IQR). The dotted line is n^3 scaled from C at n = 64.

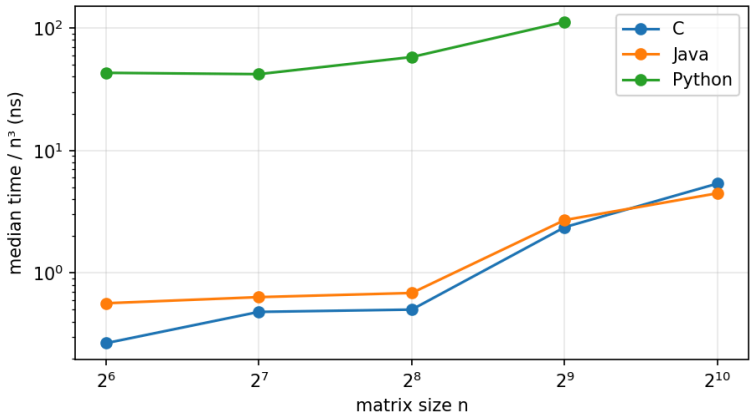


Figure 2 — Median time divided by n^3: cost of one inner-loop iteration (one multiply-add) in nanoseconds.

Variability is small for C and Java (IQR ≤ 7 % of the median); Python varies more at small n (9–11 % for n ≤ 256).

4.3 Hardware counters (perf)

`perf stat` ran for every C configuration, but in WSL2 the hardware events (cycles, instructions, cache references, cache misses) were reported as *not supported*: WSL2 does not expose the CPU performance counters to the Linux kernel. Only task-clock was available (`results/perf_c.csv`). Cache misses therefore could not be measured.

4.4 Memory and limits

Table 2 — Peak resident memory of the process (MiB) and estimated storage of A, B and C (MiB).

n	C peak RSS	Java peak RSS	Python peak RSS	arrays C/Java ($3 \cdot 8n^2$)	arrays Python ($\approx 3 \cdot 32n^2$)
64	1.9	57.1	41.0	0.09	0.4
128	2.1	57.7	42.0	0.4	1.5
256	3.1	60.2	47.8	1.5	6
512	7.8	68.8	70.3	6	24
1024	25.8	105.2	—	24	—

Limits. Largest size completed inside the budget: n = 1024 for C and Java, n = 512 for Python. Timeouts (one multiplication not finished in 20 s, `results/limits.csv`): C at n = 2048, Java at n = 2048, Python at n = 1024. The memory budget (2 GB) was never reached; the largest attempted size, 2048, needs 96 MiB of arrays in C/Java, so the limit is time, not memory.

5. Discussion

H1 — cubic growth: only partly confirmed. The time grows about ×8 per doubling only for small n (C ×8.4 from 128 to 256). From 256 to 512 it grows ×38 (C), ×32 (Java) and ×16 (Python), and Figure 2 shows the cost of one multiply-add rising from ≈ 0.5 ns to 5.4 ns in C. At n = 512 one matrix (2 MiB) exceeds the L2 cache (1.25 MiB) and the inner loop jumps 4 KiB in B at every step, so almost every access to B is a cache miss, worsened by the power-of-two stride [2]. The algorithm is $O(n^3)$, but the constant depends on the memory hierarchy. This explanation could not be confirmed with `perf`, because the hardware counters are not available in WSL2 (§4.3).

H2 — language ranking: confirmed only up to n = 512. For $n \leq 512$ C is fastest, Java 1.1–2.1× slower and Python 48–161× slower: each Python iteration runs several bytecode instructions and creates a new float object (≈ 42–58 ns against < 1 ns in C). The C–Java gap shrinks as the loop becomes memory-bound. At n = 1024 Java is 1.2× faster than C (4.80 s vs 5.76 s, IQRs not overlapping). We cannot explain this with certainty: the JIT may generate a different inner loop than GCC `-O2`, or the C process may have run on an efficiency core of the hybrid i7-1255U (no CPU affinity was set).

H3 — memory: confirmed. C's peak memory is the arrays plus ≈ 1.8 MiB of process (25.8 MiB at n = 1024 for 24 MiB of arrays). Java has a fixed cost of ≈ 57 MiB (JVM and JMH), and its peak also includes the temporary byte buffers used to read the files (105.2 MiB at n = 1024). Python starts at ≈ 41 MiB (interpreter and pytest) and grows about 5× faster per element than C, because each value is a float object (24 bytes) referenced by a list slot (8 bytes): from n = 64 to 512 it grows by 29 MiB against 6 MiB for C.

Limitations.

- One machine, under WSL2 on a Windows host: background activity can add noise (largest IQR: Python 11 % at n = 64 and 256). The results do not describe other machines.

- Hybrid CPU (performance and efficiency cores) and no CPU affinity: different runs may use different core types.
- Peak RSS includes runtime and file-loading buffers; it is not the same quantity as array storage, so the two are reported separately.
- Only 5 repetitions per configuration; the timeouts give a time limit, not a measured time.
- Hardware counters are not available in WSL2, so the cache explanation is not verified by measurement.
- C time is measured inside the program because `perf stat` only gives whole-process time; Java and Python times come from JMH and pytest-benchmark. The timing boundary (kernel only) is the same for all three.

6. Conclusions

On this machine, with the same algorithm, inputs and arithmetic, the triple-loop multiplication grows as n^3 only while the matrices fit in cache; beyond that, memory access dominates and the growth per doubling is much larger than 8. C and Java reach n = 1024 (5.8 s and 4.8 s); C is fastest up to n = 512, while at n = 1024 Java is faster for reasons we could not isolate. Pure Python is 48–161× slower than C and reaches only n = 512 in the 20 s budget. Memory follows the $8n^2$ (C, Java) or $\approx 32n^2$ (Python) storage plus a fixed runtime cost. These results are limited to these implementations, compiler and runtime settings, and hardware; they do not show that one language is faster in general.

7. References and materials

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] U. Drepper, “What every programmer should know about memory,” Red Hat, Inc., Nov. 2007. [Online]. Available: https://people.freebsd.org/~lstewart/articles/cpumemory.pdf

[3] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.

[4] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.

[5] Linux man-pages project, “proc(5) — Linux manual page.” [Online]. Available: https://man7.org/linux/man-pages/man5/proc.5.html

[6] OpenJDK, “Java Microbenchmark Harness (JMH).” [Online]. Available: https://openjdk.org/projects/code-tools/jmh/

[7] “pytest-benchmark documentation.” [Online]. Available: https://pytest-benchmark.readthedocs.io/

[8] Free Software Foundation, “Options that control optimization,” GCC online documentation. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html

[9] “perf: Linux profiling with performance counters,” perf wiki. [Online]. Available: https://perf.wiki.kernel.org/

Materials. GitHub repository: https://github.com/andreapatruno1/Assignment1_BD. It contains the source code, the README with exact build and run commands, the input generator, raw measurements (`results/raw_times.csv` and the native tool output in `results/tools/`), correctness results, environment description and the scripts that produce all tables and figures (`scripts/analyze.py`).